

kafka 기본 개념



Apache Kafka: 데이터레이크하우스의 심장

1. 카프카는 왜 만들어졌나요? (유래)

카프카는 2011년 **LinkedIn(링크드인)**에서 내부의 복잡한 데이터 흐름을 해결하기 위해 만들어졌습니다.

- **과거의 문제:** 서비스가 늘어날 때마다 서버와 DB 간의 연결(Point-to-Point)이 스파게티처럼 꼬였습니다. 데이터 하나를 수정하면 연결된 모든 곳을 고쳐야 했죠.
- **해결책:** 모든 데이터를 한곳으로 모으고 필요한 곳에서 가져가는 **'거대한 데이터 파이프라인'**을 만들자! 해서 탄생한 것이 카프카입니다.



Apache Kafka의 핵심 동작 원리

카프카를 지탱하는 3가지 설계 철학은 **"로그(Log)", "분산(Distributed)", "순차적 처리(Sequential)"**입니다.

1. 로그 기반의 저장 방식 (Append-only Log)

카프카는 데이터를 데이터베이스처럼 수정하거나 삭제하지 않습니다. 들어오는 데이터를 파일 끝에 단순히 이어 붙이는(Append) **로그 파일** 형태를 유지합니다.

- **원리:** 파일 끝에 붙이기만 하면 되기 때문에 디스크 쓰기 속도가 메모리만큼 빠릅니다 (Sequential I/O).
- **결과:** 데이터가 쌓여 있어도 성능 저하가 거의 없습니다.

2. 토픽(Topic)과 파티션(Partition)

카프카는 데이터를 **토픽**이라는 주제로 나누고, 이를 다시 여러 개의 **파티션**으로 쪼개서 분산 저장합니다.

- **원리:** 한 대의 서버가 감당하기 힘든 대량의 데이터를 여러 서버(Broker)에 나누어 담습니다.

- **병렬 처리:** 파티션이 여러 개이므로, 컨슈머(Consumer) 여러 대가 동시에 데이터를 읽어들일 수 있어 처리량이 기하급수적으로 늘어납니다.

3. 오프셋(Offset) 관리

파티션 내부에서 각 메시지는 고유한 번호인 ****오프셋(Offset)****을 가집니다.

- **원리:** 컨슈머는 "내가 몇 번 오프셋까지 읽었어"라는 정보만 기록합니다.
- **장점:** 컨슈머가 죽었다 살아나도 기록된 오프셋 다음부터 읽으면 되므로 데이터 유실이나 중복 처리를 방지할 수 있습니다.

4. 복제와 고가용성 (Replication)

카프카는 파티션을 여러 대의 브로커에 복제해둡니다.

- **원리:** 하나는 **리더(Leader)**, 나머지는 ****팔로워(Follower)****가 됩니다.
- **고가용성:** 리더 브로커가 고장 나도 팔로워 중 하나가 즉시 리더가 되어 서비스를 중단 없이 이어갑니다.

다른 곳에서는 어떤 용도로 쓰나요?

단순히 데이터를 옮기는 것 이상으로 다양하게 쓰입니다.

- **실시간 로그 및 이벤트 수집:** 웹사이트 클릭 로그, 앱 활동 내역을 실시간으로 긁어모아 분석합니다.
- **마이크로서비스(MSA) 간 통신:** 서비스 A가 작업 완료 메시지를 카프카에 던지면, 서비스 B가 그걸 보고 다음 작업을 시작하는 방식(이벤트 주도 설계)입니다.
- **금융권 실시간 이상거래 탐지(FDS):** 결제 데이터가 들어오는 즉시 카프카를 거쳐 실시간 분석 엔진으로 보내 사기 결제인지 판별합니다.
- **데이터레이크하우스 구성 (질문자님 사례):** 여러 소스(DB, 로그)에서 데이터를 긁어와 MinIO나 HDFS 같은 저장소로 안전하게 배달하는 '우체부' 역할을 합니다.

장점

1. **압도적인 속도:** 디스크에 순차적으로 기록하는 방식을 써서 쓰기 속도가 엄청나게 빠릅니다.

2. **영속성 (Persistence):** 데이터를 일정 기간 저장해두기 때문에, 장애가 발생해도 과거 시점부터 다시 데이터를 처리할 수 있습니다.
3. **결합도 분리 (Decoupling):** 데이터를 주는 쪽(Producer)과 받는 쪽(Consumer)이 서로 몰라도 카프카만 통하면 됩니다.

✖ 단점

1. **운영 난이도:** 오늘 경험하신 것처럼 GUI가 불친절하고, 설정(Partition, Replica 등)이 복잡해 공부할 게 많습니다.
2. **추가 인프라 비용:** 카프카를 관리하기 위해 주키퍼(Zookeeper)나 KRaft 같은 추가 시스템이 필요합니다.
3. **작은 데이터 위주:** 영화 한 편 같은 거대 파일 전송보다는 **작고 수많은 이벤트 메시지** 전송에 최적화되어 있습니다.

비교 항목	기존 기술 (Oracle, RabbitMQ)	Apache Kafka
처리량 (Throughput)	대량 데이터 처리 시 병목 발생	초당 수백만 건 처리 가능
데이터 보관	소비(Consume)하면 데이터가 사라짐	디스크에 저장 되어 재처리 가능
확장성	서버 사양을 높여야 함 (Scale-up)	서버를 옆으로 계속 붙임 (Scale-out)
신뢰성	단일 장애점 발생 시 위험	데이터를 여러 곳에 복제 (Replication)